

Chest-Pain Triage Chat Agent

1. Problem Statement

A multi-turn conversational agent that triages adult patients with non-traumatic chest pain into one of the protocol's **seven defined dispositions**.

The system uses a **hybrid architecture**:

- A **deterministic rules engine** that exclusively owns and makes every triage decision.
- A **local, open-source LLM** that is used only to:
 - Extract structured facts from patients' free-text responses.
 - Classifies how distressed the patient sounds.
 - Phrase appropriate responses..

The LLM does not diagnose patients or make triage or disposition decisions.

2. Pipeline architecture

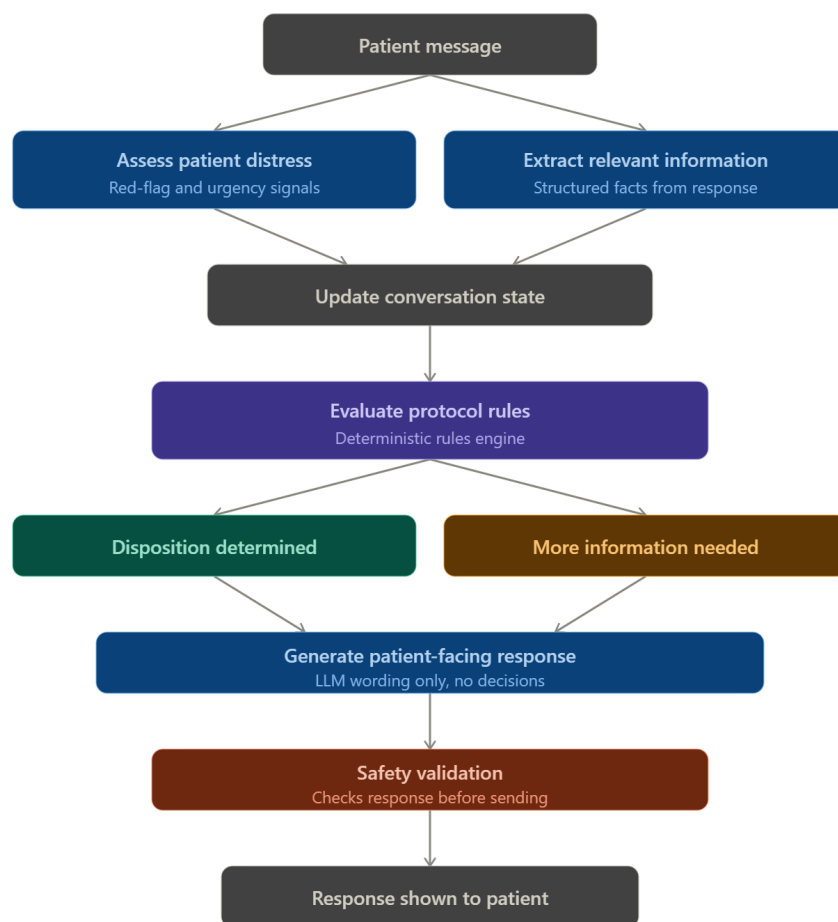
The system separates language understanding from clinical decision-making.

Patient messages are interpreted by the LLM, while all triage decisions are made by a deterministic rules engine. The rules engine never reads raw patient text directly, and the LLM never independently determines a disposition.

This separation ensures that the safety-critical **routing decision** remains **deterministic**, **testable**, and **traceable** to a specific protocol rule

Component	Responsibility	LLM?
Protocol Rule Table	The guideline as an ordered decision table 7 dispositions ~30 rules	no
Decision Engine	Evaluates every rule against known facts using TRUE /FALSE / UNKNOWN logic;	no
Conversation State	Accumulates the facts turn by turn. Its absences are what mark a fact "unknown", which drives the three-valued logic and tells Dialogue Control what to ask next. Re-fed to the Decision Engine every turn.	no
Dialogue Control	Picks the next question (the most-urgent still-possible rule needs).	no

	Escalates immediately stops asking when a rule fires.	
Information Extractor	Converts patient text into structured facts against a fixed schema and validates and drops anything off-schema	yes
Distress Assessment	Reads acute-distress cues from how the patient sounds, separately from the facts they report	yes
Response Phrasing	Phrases the next question naturally for a worried adult; the final disposition sentence is fixed protocol text, not generated.	yes
Safety validation	Post-filters outgoing text. Blocks diagnosis language	no

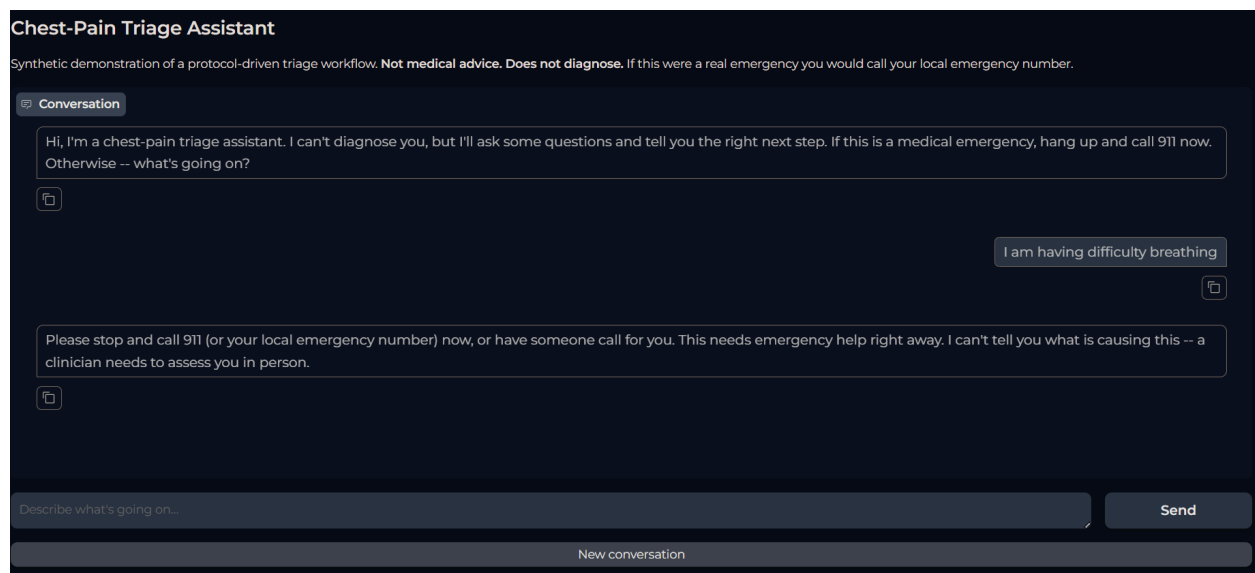


Why this architecture?

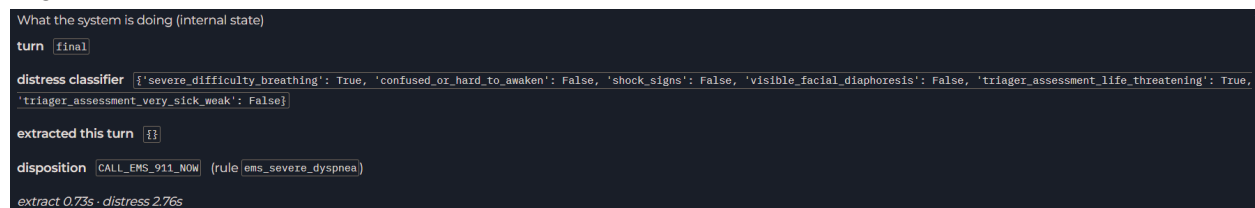
- This separation keeps the safety-critical disposition decision **deterministic** and independent of the LLM.

- The rules engine is **unit-testable** rule by rule, and each decision can be reproduced from the saved rules version and patient state for post-incident review.
- Because the rules engine operates on **structured information** rather than raw language, the final decision is not directly influenced by paraphrasing or prompt injection.
- The LLM can still make mistakes when extracting or interpreting information, but those errors remain identifiable and separately testable rather than being hidden inside an end-to-end, unexplainable model decision.

2.1 Chat Interface



"What the system is doing" panel tells us the: extracted slots, distress flags, the rule that fired, per-call latency, which is the visual proof that the disposition comes from the deterministic engine, not the model.



3. Model selection

Llama 3.1 8B Instruct was chosen for fit to this task rather than for general capability:

- The task is schema-shaped extraction and templated phrasing, not open-ended reasoning. Llama's one benchmark advantage, is the closest proxy to what this pipeline actually needs.
- Because the decision logic lives entirely in the deterministic rules engine, the LLM's job is bounded: fill known fields correctly, don't hallucinate unset ones, and phrase output. This narrows the capability gap between candidate models relative to general benchmarks, which test breadth that isn't needed.
- Validated that the chosen model meets the task's requirements (`extraction_golden.jsonl`, `scenarios.jsonl`, full conversational evaluations) rather than relying on third-party leaderboards, since task-specific fit is what matters here.

4. Synthetic datasets

file	rows	one row is	role
data/eval/scenarios.jsonl	59	A full synthetic case. Patient's complete ground-truth facts, plus the disposition and the exact protocol rule it must reach.	end-to-end evaluation
data/eval/extraction_golden.jsonl	112	One patient sentence + the exact structured fields a perfect extractor should return from it	isolated extraction evaluation
data/train/sft_v1.jsonl	290	one fine-tuning example. A production-identical prompt (system + user) paired with the ideal model output, for either the extraction or the distress call	SFT/LoRA training
data/eval/extraction_val.jsonl / _test.jsonl	25 / 87	the 112 golden rows split by template (tag), so no phrasing of a template lands on both sides.	validation / held-out test

4.1 How the eval data is labelled

Each scenario's gold disposition is derived **deterministically** — its ground-truth facts run through the same `rules.yaml`, spot-checked by hand against the protocol PDF.

The simulated patient (patient_sim.py) is deterministic and templated: it converts facts into short patient-style sentences, volunteering red flags first (as a distressed caller would), then answering exactly what the agent asks. This makes per-turn latency measurable with no simulated-patient latency to subtract.

5. Evaluation — baseline

code/eval_harness.py drives a full simulated conversation per scenario and scores the final disposition.

Model: meta-llama/Llama-3.1-8B-Instruct, vLLM on a single GPU,
prefix caching on, streaming on, temperature 0 for extraction/distress.

5.1 Accuracy (59 scenarios)

metric	value	definition
triage accuracy	0.915	exact disposition == gold
workflow accuracy	0.915	(disposition, rule_id) == gold . "right answer, right reason"
under-triage rate	0.034	predicted disposition less urgent than gold
over-triage rate	0.051	predicted disposition more urgent than gold
red-flag recall (gold = 911)	0.944	of gold CALL_EMS_911_NOW, fraction predicted there (17/18)
red-flag recall (gold ∈ {911, ED-now})	0.970	fraction kept in the top-2 tier (32/33)
did not terminate	0	conversations that hit the turn cap without a disposition

Plain accuracy is **not** the headline: error cost is asymmetric (an under-triaged MI ≠ an over-triaged muscle strain) and classes are imbalanced.

Red-flag recall and under-triage rate are the metrics that matter; target red-flag recall ≥ 0.99 .

5.2 Latency

	mean	p50	p90
call TTFT (s) — GPU	0.79	0.32	2.15
call total (s) — GPU	3.37	3.38	6.51
turns / scenario	6.4	7	10
wall-clock / scenario (s) — GPU	52.0	52.6	77.6

(Wall-clock per scenario includes ~7 sequential turns × 2 LLM calls each, run 8-way concurrently across scenarios.)

Per-turn latency is almost entirely the two LLM calls; the engine and state machine are microseconds. Extraction is the larger call (input carries the running context); the distress call is short.

5.3 Fail Cases

In offline mode, where a perfect oracle supplies the facts, the rules engine reaches the correct disposition on all 59 scenarios.

So in every miss below the engine applied the right rule to the facts it was given. The error is upstream, in fact the LLM produced.

Four are distress-classification errors, one is an extraction error.

scenario	gold → predicted	cause	call
ems: triager judges life-threatening	CALL_EMS_911 → HOME_CARE	Did not raise triager_assessment_life_threatening on the opening message. Even a turn-1 raise is cleared by later calm turns.	distress
educ: triager judges very sick/weak	GO_TO_ED_UCC → HOME_CARE	Same pattern — triager_assessment_very_sick_weak is raised on turn 1, then erased by a later turn	distress
ed: difficulty breathing (non-severe)	GO_TO_ED_NOW → CALL_EMS_911	Graded a mild "a bit short of breath" as severe_difficulty_breathing = true, which fires the EMS rule instead of	distress

		the ED rule.	
educ: pleuritic pain	GO_TO_ED_UCC → CALL_EMS_911	classifier read "hurts to breathe in" as a breathing emergency	distress
pcp3: nitro-resolved	SEE_PCP_3_DAYS → GO_TO_ED_NOW	Invented other_symptoms = [difficulty_breathing] from the vague answer "yeah, some of those", which fires ed_difficulty_breathing.	extract

6. Fine-tuning — baseline vs LoRA

6.1 Method

	value
technique	QLoRA (4-bit nf4, double quant, bf16 compute)
rationale	full fine-tune of 8 B won't fit training state in 24 GB; QLoRA trains comfortably and vLLM serves the adapter unmerged; quality gap vs full LoRA is negligible for a task this narrow
LoRA config	$r = 16$, $\alpha = 32$, dropout 0.05, target q_proj,k_proj,v_proj,o_proj
trainable params	13.6 M / 8.04 B (0.17 %)
optimizer / schedule	paged_adamw_8bit, cosine, warmup 0.03, lr 2e-4
batch	$2 \times$ grad-accum 8 (effective 16)
max-seq-len	2048 (extraction rows peak ~1,800 tokens)
v1 (reported)	SFT set, 290 rows, 3 epochs, final train loss 0.662
v2 (rolled back)	true-only distress output format + 5 epochs → regressed the distress classifier (isolated micro-F1 $\approx$ 0.29, optional-flag recall → 0) and over-fit the extractor (guard-slot key-F1 1.0 → 0.67). Format + epoch changed together, so

	cause not isolated; reverted in the dataset generator.
--	--

6.2 Evaluation approach

The disposition is owned by the deterministic engine, so LoRA is **not** expected to move end-to-end triage accuracy much.

What it should improve is what it touches: slot extraction and distress classification.

Those are measured **in isolation** :

- code/eval_extraction.py (112 golden rows through extract_slots()) and
- code/eval_distress.py (59 chief-complaint utterances through classify_distress()) . So the number reflects the model, not the pipeline.

End-to-end accuracy is reported as a **secondary** number.

6.3 Results

Isolated slot extraction — 112 golden rows

metric	base	LoRA v1	Δ	95% CI on Δ
row exact-match	0.562	0.607	+0.045	[−0.036, +0.125]
key precision	0.768	0.782	+0.015	[−0.056, +0.090]
key recall	0.790	0.833	+0.043	[−0.022, +0.115]
key F1	0.779	0.807	+0.028	[−0.030, +0.098]
value accuracy	0.844	0.826	−0.018	[−0.080, +0.044]
hallucination row-rate	0.241	0.214	−0.027	[−0.098, +0.045]

Isolated distress classification — 59 utterances, 6 flags

metric	base	LoRA v1	Δ	95% CI on Δ
micro precision	0.636	0.571	−0.065	[−0.208, +0.069]
micro recall	0.875	1.000	+0.125	[0.000, +0.417]

micro F1	0.737	0.727	-0.010	[-0.156, +0.196]
flag exact-match	0.915	0.915	0.000	[-0.051, +0.051]

End-to-end (secondary) — 59 scenarios

metric	baseline	LoRA v1	note
triage accuracy	0.915	0.915	within baseline CI [0.83, 0.98]
workflow accuracy	0.915	0.915	Same, neutral
red-flag recall (911)	0.944	0.889	within baseline CI [0.81, 1.00]
under-triage rate	0.034	0.051	within baseline CI [0.00, 0.09]
over-triage rate	0.051	0.034	within baseline CI [0.00, 0.12]
call total (s), p50	3.38	2.31	

Analysis

Positives:

- Direction is consistent and on-target. Every coverage metric moved up. Row exact-match +4.5 pts, key recall +4.3, key F1 +2.8, distress recall+12.5 and the hallucination row-rate moved down 2.7. The SFT set was built to recover missed slots and missed flags, and that is where things moved.
- Distress recall reached 1.000 : the adapter missed no required red-flag across all 59 scenarios (base missed one). Not significant, but "never miss a needed escalation flag" is the right failure mode for triage.
- End-to-end stayed stable: identical triage and workflow accuracy, and over-triage actually fell (5.1% → 3.4%). Fine-tuning did not destabilise the pipeline.
- Generation is ~30% faster (call p50 3.38 s → 2.31 s): the model learned a tighter output shape.

Cons:

- Nothing is significant. At this sample size a genuine 3-point F1 gain would be undetectable; This is a design limit of a 112-row eval, not just a property of the adapter.
- Precision consistently leans down. Small on extraction, larger on distress (-6.5 pts, spurious-flag count 4 → 5). Not significant, but over-flagging is the wrong direction for a triage system, and it's the one place the adapter looks worse.

- The recall gain does not reach a disposition: even a correctly-raised flag is erased downstream before it changes routing. The metric that improved is decoupled from the outcome that matters.

Net assessment: The LoRA fine-tune moves in the right direction. Broader coverage, faster inference, no evidence of regression. but none of this can be confirmed at a sample size this small.

6.4 Pipeline defect that gates the end-to-end result

classify_distress runs on every turn, and its schema forces *life_threatening* and *very_sick_or_weak* to be booleans. So every turn emits one, whether or not that turn carries any signal.

There's no notion of "**unset**" here, only the latest boolean. A patient who opens in panic "this is an emergency" and then answers the follow-up questions calmly will correctly **flag life_threatening=True** on **turn 1** and just as correctly flag it **False** on **turn 3**, since turn 3 genuinely contains no urgency language.

The later value **overwrites** the earlier one, and the scenario falls through to HOME_CARE, not because the classifier failed, but because the pipeline forgot it had ever succeeded.

Evidence. Running *code/probe_distress.py --scenarios* on the real turn-1 utterance for ems: triager judges life-threatening; the baseline model misses it entirely (all-false), while LoRA v1 correctly returns *life_threatening=True*. The scenario still ends in HOME_CARE. The flag was raised, and then cleared.

Attempted fix, reverted. Making the six observed flags monotonic. Once True, never False again within a conversation, was the obvious patch. It broke something else: baseline over-triage jumped from **0.051** to **0.339**. The overwrite that looked like a bug was also silently walking back roughly 15 classifier false positives per run. The classifier isn't precise enough for blanket stickiness to be safe, making early flags permanent means every transient false positive becomes a permanent one too.

The real fix: running the distress classifier once, on the chief-complaint turn, rather than every turn; or gating stickiness on classifier confidence rather than treating all positives equally, is scoped as future work.

Why does this limit what fine-tuning can do?

Two of the five observed failures trace to this overwrite, not to model quality.

The schema demands a fresh boolean on every turn, and the pipeline keeps only the last one. No adapter, however well it classifies any single turn, can fix a bug in how those classifications are combined across turns.

7. Safety behaviour

safety rule (brief)	how enforced	evidence
Must not diagnose	disposition sentences are templated from the Decision (DISPOSITION_SCRIPT); rule_out (differential) is never surfaced to the patient; guardrails.py post-filters free text for diagnosis language and unbacked reassurance	tests/test_guardrails.py
Stop questioning and escalate on a red flag	state_machine._next_turn finalizes the instant a rule fires; at most one exception-confirmation question and (for ED tiers only) a pregnancy question follow; EMS-911 finalizes immediately	tests/test_state_machine.py, tests/test_reachability.py
Never assume acid reflux / heartburn	GERD is reachable through exactly one rule (cpcp24_gerd) requiring all three of previously-diagnosed heartburn match + burning + sour taste; suspected_cause is decision_relevant=False and can terminate nothing; EXTRACT_SYS rule 7 forbids filling the GERD slots from "it's just my reflux"	tests/test_decision_engine.py (asserts 2-of-3 does not conclude GERD)

8. Limitations & next steps

1. **Distress-flag overwrite defect:** highest-value fix; expected to recover 2–3 failures with no fine-tuning and lift red-flag recall toward 1.0.

2. **Validation split created late** : data/eval/extraction_val.jsonl (25 rows/ 20 tags) and extraction_test.jsonl (87 / 77), split by tag via code/split_val.py, exist now for future selection. The v1-vs-baseline comparison predates it and uses the full 112-row golden set. since v1 shows no measurable effect either way, this does not change the conclusion.
3. **Thin eval** : 59 scenarios, 112 golden rows, deterministic templated patient (patient_sim.py). Several distress flags have support = 1. CIs are wide (triage accuracy ± 8 pts). A larger, non-templated eval. Ideally de-identified real transcripts with clinician-adjudicated labels, is the honest next step, and the setting in which the extraction-recall lean might become significant.
4. **Smaller model** : with extraction schema-shaped and NLG templated, Phi-3.5-mini or Qwen2.5-3B may match Llama-8B at lower latency; the comparison row is not yet run.